Assignment: Arrays and Linked Lists [deadline 11th March 2026 Wednesday before 2pm]

Q1. Array vs Linked List — Technical Comparison

Write a detailed comparison between **arrays** and **linked lists** based on the following:

1. Memory layout
2. Access time
3. Insertion and deletion
4. Cache friendliness
5. Memory overhead
6. Searching
7. Real-world use cases

Also answer:
Why is random access efficient in arrays but not in linked lists?

Q2. Implement Basic Array Operations

Write a program to perform the following operations on an array:

- traverse
- insert at beginning
- insert at end
- insert at any position
- delete from beginning
- delete from end
- delete from any position
- search an element

Concept focus: indexing, overflow, shifting cost

Q3. Dry Run of Array Insertion

Given the array:

[10, 20, 30, 40, 50]

Capacity = 10, Current size = 5

Show step by step how the array changes when:

1. Insert 5 at index 0
2. Insert 25 at index 2
3. Insert 60 at the end

Also state the time complexity of each insertion.

Q4. Find Second Largest and Second Smallest in an Array

Write a program to find:

- largest
- second largest
- smallest
- second smallest

in a single traversal if possible.

Then explain:

- what happens when all elements are same?
- what happens when duplicates exist?

Example: [8, 2, 5, 2, 9, 9, 1]

Q5. Reverse an Array In-Place

Write a function to reverse an array **without using extra array space**.

Then answer:

- What is the time complexity?
- Why is this called an in-place algorithm?
- Can this be done recursively?

Part B — Linked List Fundamentals

Q6. Implement a Singly Linked List

Create a singly linked list and implement:

- insert at beginning
- insert at end
- insert after a given value
- delete from beginning
- delete from end
- delete a specific value
- display list
- count nodes

Then explain the role of:

- head
- next
- dynamic memory allocation

Q7. Trace Singly Linked List Insertions and Deletions

Start with an empty linked list and perform:

1. Insert 10 at beginning
2. Insert 20 at end
3. Insert 5 at beginning
4. Insert 15 after 10

5. Delete 20
6. Delete beginning node

Draw the linked list after each step using node diagrams like:

[5|*] -> [10|*] -> [15|null]

Q8. Array Deletion vs Linked List Deletion

Suppose you want to delete the element at the start, middle, and end.

Compare deletion in:

- array
- singly linked list

For each case, explain:

- required operations
- time complexity
- why one is better than the other

Q9. Search in Singly Linked List

Write a program to search for a given key in a singly linked list.

Return:

- whether found or not
- node position
- total comparisons made

Then compare it with linear search in arrays.

Q10. Count Even, Odd, Positive, and Negative Nodes

Given a singly linked list of integers, write a program to count:

- even numbers

- odd numbers
- positive numbers
- negative numbers
- zero values

Also display the final counts.

Part C — Intermediate Algorithms

Q11. Reverse a Singly Linked List

Write a program to reverse a singly linked list iteratively.

Then answer:

1. Why do we need three pointers?
2. What happens if we lose the next node reference?
3. Write the recursive version as well.

Concept focus: pointer manipulation

Q12. Find the Middle Node of a Linked List

Write two approaches to find the middle node:

1. By counting total nodes
2. By slow and fast pointer technique

Then compare both approaches.

Also answer:

- If the list has even number of nodes, which middle will you return?
- Why is slow/fast pointer considered elegant?

Q13. Detect a Cycle in a Linked List

Write a program to detect whether a singly linked list contains a loop/cycle.

Explain:

- how slow and fast pointers work
- why they meet if a cycle exists
- time and space complexity

Q14. Merge Two Sorted Arrays

Given two sorted arrays, merge them into a third sorted array.

Then do the same problem again with this restriction:

- **Do not use extra third array**
- discuss whether fully in-place merging is easy or difficult in arrays

Example:
A = [1, 4, 7, 10]
B = [2, 3, 9]

Q15. Merge Two Sorted Linked Lists

Given two sorted singly linked lists, merge them into a single sorted linked list.

Then answer:

- Why is merging more natural in linked lists than arrays?
- Can this be done without creating new nodes?
- What is the complexity?

Part D — Advanced Practice and Reasoning

Q16. Remove Duplicates

Solve duplicate removal in both:

Part A: Sorted Array

Remove duplicates from a sorted array.

Part B: Sorted Linked List

Remove duplicates from a sorted linked list.

Then answer:

- Why is duplicate removal easier in sorted structures?
- How would the problem change for unsorted data?

Q17. Rotate Array and Rotate Linked List

Perform left rotation by k positions on:

1. an array
2. a singly linked list

Then compare both problems:

- which is easier?
- which is more efficient?
- how does the implementation differ?

Example:
Input: [1,2,3,4,5,6,7], k = 2
Output: [3,4,5,6,7,1,2]

Q18. Find Nth Node from End

Write a program to find the **nth node from the end** in a singly linked list.

Solve using:

1. length counting method
2. two-pointer method

Then explain why the two-pointer method is preferred.

Q19. Design a Dynamic Array Class

Implement your own simplified **dynamic array** class in C++ with:

- internal pointer

- current size
- capacity
- resize when full
- append
- insert
- delete
- get element
- set element

Then answer:

- How is a dynamic array different from a normal array?
- Why is resizing expensive?
- Why is append often considered amortized O(1)?

Q20. Mini Problem-Solving Challenge: Choose the Right Data Structure

For each scenario, decide whether **array** or **linked list** is more suitable. Justify your answer technically.

1. Store marks of 1000 students where random access is frequent
2. Maintain a music playlist where songs are inserted/deleted often
3. Implement browser history with back/forward navigation
4. Store image pixels in matrix form
5. Implement undo/redo in editor
6. Real-time insertion of train stations between two stations
7. Store fixed monthly sales data
8. Sparse data with unpredictable growth

Your answer must include:

- chosen structure

- reason
- time complexity considerations
- memory implications

Bonus Challenge Questions

Bonus 1. Implement a Doubly Linked List

Implement:

- insertion at beginning/end
- deletion from beginning/end
- traversal forward and backward

Then explain how a doubly linked list differs from a singly linked list.

Bonus 2. Implement a Circular Linked List

Write code for:

- insertion
- traversal
- deletion

Then explain one real-world use case of circular linked list.

Bonus 3. Polynomial Representation

Represent a polynomial using:

1. array
2. linked list

Then discuss which representation is better for sparse polynomials and why.

Suggested Submission Requirements

Submit the following for each coding question:

1. Problem statement
2. Algorithm / logic
3. Source code
4. Dry run with sample input
5. Time complexity
6. Space complexity
7. Final output screenshot